Try and Catch

DATA TYPES AND EXCEPTIONS IN JAVA

Jim White
Java Developer

Problems in Java

- Problems or issues can occur in applications
  - Due to wrong input
  - Due to coding errors
  - Due to unforeseen circumstances
- Problems or issues are called *exceptions*
  - Interrupt the normal control flow
- Java *throws* exceptions

Exception Example

```
public static void main(String[] args) {
    ArrayList<String> allStars = new ArrayList<String>();
    allStars.add("Mays");
    allStars.add("Aaron");
    allStars.add("Ruth");
    String last = allStars.get(3); // This will cause an exception
    System.out.println(last);
}
```

```
Exception in thread "main" java.lang.IndexOutOfBoundsException: Index 3 out of
bounds for length 3
```

Try

- Wrap exception-generating code with a try-block
  - "Try this code and if there is a problem pass it to the catch block"

```
try {
    // Statements to be executed and watched for exceptions
}
```

Catch

- Place a catch-block after the try
  - Acts as a *handler* to address exception problems
  - The *exception* caught is passed into this block as a parameter
- try/catch blocks are required for some types exceptions
  - Optional for others

```
try {
    // Statements to be executed and watched for exceptions
} catch (<Exception type> <variable name>) {
    // Statements to execute if an exception has occurred
}
```

Catching IndexOutOfBoundsException

```
try {
  ArrayList<String> allStars = new ArrayList<String>();
  allStars.add("Mays");
  allStars.add("Aaron");
  allStars.add("Ruth");
  String last = allStars.get(3); // Accessing an element that isn't there
  System.out.println(last);
} catch (IndexOutOfBoundsException e) {
  System.out.println("Oops - wrong index");
}
```

```
Oops - wrong index
```

Multiple Catch

- Use multiple `catch` blocks to handle different types of exceptions
- Each `catch` gets its own exception type and exception variable
- When exception occurs, Java checks the `catch` blocks in order until it finds a match

```
try {
    ArrayList<String> nums
      = new ArrayList<String>();
    nums.add("one");
    String last = nums.get(0);
    Integer.valueOf(last);
    System.out.println(last);
} catch (IndexOutOfBoundsException eIndex) {
    System.out.println("Oops - wrong index");
} catch (NumberFormatException eValueOf) {
    System.out.println(
      "Oops - String not a number");
}
```

```
Oops - String not a number
```

Catch "All"

- Use `catch (Exception variable){ }` as a catch all
  - Handles any exception not handled by the other catch blocks

```
try {
    // Code to try
} catch (IndexOutOfBoundsException eIndex) {
    System.out.println("Oops - wrong index");
} catch (NumberFormatException eValueOf) {
    System.out.println("Oops - String not a number");
} catch (Exception e) { // Handles all exceptions except the two above
    System.out.println("Something unplanned happened");
}
```

Finally

- Code in `finally` block always executes

```
// When exception happens
try {
    Integer.valueOf("one");
} catch (NumberFormatException eValueOf) {
    System.out.println("Oops");
} finally {
    System.out.println("Doing cleanup");
}
```

```
Oops
Doing cleanup
```

```
// When no exception gets thrown
try {
    Integer.valueOf("1");
} catch (NumberFormatException eValueOf) {
    System.out.println("Oops");
} finally {
    System.out.println("Doing cleanup");
}
```

```
Doing cleanup
```

Exception object

- The `Exception` object passed into the catch block is rich with information
- Use methods on the object to learn more about what went wrong
  - `.getMessage()` to get the human readable cause
  - `.getClass()` to get the Exception class or type name
  - `.printStackTrace()` to display the stack trace (next slide)

```
public class ExceptionExample {

    public static void main(String[] args) {
        try {
            Integer.valueOf("one");
        } catch (NumberFormatException e) {
            System.out.println(e.getMessage());
            e.printStackTrace();
        }
    }
}
```

Exception stack trace

- The **stack trace** provides a record of the execution flow to the problem
  - Below: output from the previous code example - including the printed *stack trace*

```
For input string: "one"
java.lang.NumberFormatException: For input string: "one"
    at java.base/java.lang.NumberFormatException.forInputString(NumberFormatException.java:67)
    at java.base/java.lang.Integer.parseInt(Integer.java:661)
    at java.base/java.lang.Integer.valueOf(Integer.java:988)
    at foo/exceptions.ExceptionExample.main(ExceptionExample.java:7)
```

Let's practice!

DATA TYPES AND EXCEPTIONS IN JAVA

Errors and Exceptions

DATA TYPES AND EXCEPTIONS IN JAVA

Jim White
Java Developer

datacamp

Exceptions vs Errors

- `Error` is a serious issue or problem
- Applications cannot handle and recover from errors
  - Errors cause the application to "crash" or stop
  - Example: an out of memory problem is an error in Java
- Applications can handle and recover from exceptions
  - Try to access elements outside the array size is an exception.

Example Error

```
import java.util.*;

public class CauseOutOfMemory {
    public static void main(String[] args) {
        List<Long> numbers = new ArrayList<Long>();
        long counter = 0;
        while (true) {
            numbers.add(counter);
            counter++;
        }
    }
}
```

```
Exception in thread "main" java.lang.OutOfMemoryError: Java heap space
    at java.base/java.lang.Long.valueOf(Long.java:1204)
    at foo/exceptions.CauseOutOfMemory.main(CauseOutOfMemory.java:12)
```

Two types of exceptions

	Checked Exception	Runtime ("unchecked") Exception
Require handling	Yes	No
General cause	Things outside our control	Our programming mistakes
Recovery	Can usually recover when anticipated	Usually unrecoverable
Recovery mechanism	try/catch or throws	Best to code carefully to avoid them
Example exception	File not found	Array index out of bounds

Checked and Runtime exceptions

- `Exception` subclasses not also subclasses of `RuntimeException` are **checked exceptions**
  - Must be dealt with (i.e. handled)
  - Ex: file not found (`FileNotFoundException`)
- `RuntimeException` superclass for all **unchecked exceptions**
  - Do not require handling
  - Ex: accessing an array index that does not exist (`IndexOutOfBoundsException`)
  - Ex: trying to divide by zero (`ArithmeticException`)

RuntimeException

RuntimeException Subclass	Description of when thrown
ArithmeticException	Bad arithmetic calculations like dividing by 0
IndexOutOfBoundsException	Index used on an array, String, etc. is out of range
NegativeArraySizeException	Trying to create an array with a negative size

```
int y = 5/0; // ArithmeticException causing code
int[] list = new int[-1];  //NegativeArraySizeException causing code
```

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
Exception in thread "main" java.lang.NegativeArraySizeException: -1
```

[1] See https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/RuntimeException.html

"Checked" Exception

```
public class LoadClass {
    public static void main(String[] args) {
        Class myClass = Class.forName("com.mysql.Driver");
    }
}
```

```
LoadClass.java:3: error: unreported exception ClassNotFoundException;
must be caught or declared to be thrown
                Class myClass = Class.forName("com.mysql.Driver");
                                              ^
1 error
```

[1] See https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Exception.html

Let's practice!

DATA TYPES AND EXCEPTIONS IN JAVA

Throwing

DATA TYPES AND EXCEPTIONS IN JAVA

Jim White
Java Developer

Handling options

- Checked exceptions require "handling"
  - "Handling" runtime exceptions is optional
- Two options for "handling"
  - Use a try and catch to capture the `Exception` and write code to address the issue
  - "Throw" the exception
- Try/catch is preferred means to handle an exception
  - Not always possible and sometimes better to consolidate exception handling

Throws keyword

- Use `throws` on any method to pass the exception back to the calling method
  - Versus handling the exception in the method
  - Called "throwing an exception" or "passing-the-buck"
  - Passing the responsibility for handling the exception to a caller
- Use a comma separated list of exception types after `throws`
  - Indicates what types of exceptions the caller method might anticipate

```
public static void someMethod() throws IndexOutOfBoundsException,
NegativeArraySizeException, NullPointerException {
  // method code
}
```

Throws example

Normal try-catch to handle an `Exception`

```
public static void main(String[] args) {
  someMethod();
}
public static void someMethod(){
  try {
    ArrayList<String> games
      = new ArrayList<String>();
    games.add("Monopoly");
    games.add("Chess");
    games.get(3);
  } catch (IndexOutOfBoundsException e) {
    System.out.println(
   "Oops - trying to get non-existent item");
  }
}
```

Throws to handle an `Exception`

```
public static void main(String[] args) {
  try {
    someMethod();
  } catch (IndexOutOfBoundsException e) {
    System.out.println(
   "someMethod tried to get non-existent item");
  }
}
public static void someMethod()
      throws IndexOutOfBoundsException {
  ArrayList<String> games
    = new ArrayList<String>();
  games.add("Monopoly");
  games.add("Chess");
  games.get(3);
}
```

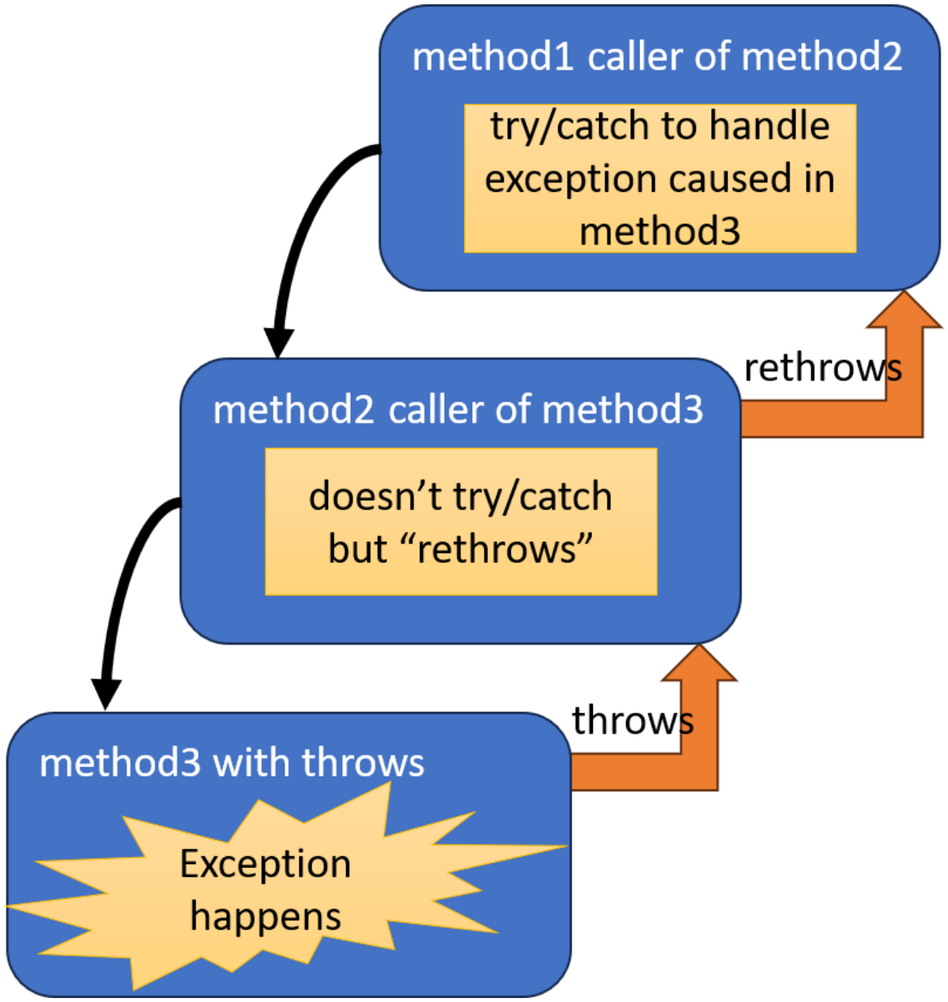
When to use throws

- "Passing-the-buck" or throwing an exception is something we try to avoid
  - Better practice is, usually, to try-catch exceptions where they happen
- Reasons for throwing rather than using try-catch include:
  - The method where the exception occurs may not know how to recover
  - Centralizing the handling to reduce redundant try/catch handling code

[1] Photo from https://unsplash.com/@philipparosetite

Catch and rethrow

- An `Exception` can be "**rethrown**"
  - A calling method also calls throws instead of having a try/catch
- Rethrowing from `main` will cause the application to stop



Rethrowing example

```
public class RethrowExample {
    public static void main(String[] args) {
        try {
            method1(0);
        } catch (ArithmeticException e) {
            System.out.println("Oops - tried a bad quotient");
        }
    }
    public static void method1(int divisor) throws ArithmeticException {
        method2(divisor);
    }
    public static void method2 (int divisor) throws ArithmeticException {
        int z = 5/divisor;
        System.out.println(z);
    }
}
```

Let's practice!

DATA TYPES AND EXCEPTIONS IN JAVA

Logging

DATA TYPES AND EXCEPTIONS IN JAVA

Jim White
Java Developer

What is a log

- Applications are made up of many lines of code
  - Apps contain a lot of control flow (if/then/else, for loops, while loops, etc.)
  - Apps contain a lot of classes and methods
  - Tracing through all the code can challenging
- A **log** serves as an application diary
  - Documenting what code has been invoked
  - Documenting where problems/issue have occurred

Logging

- Java comes with built-in logging capability
- We control what text goes to into the log
  - Similar to how we call on `System.out.println()` to leave messages in the console
- Logging is used to record what's happened
  - This includes code executed, problems encountered, and resource (like a database) used
- Logging assists developers monitor and understand application activity for debugging
- Recorded log messages can be stored in a file, database, or other system
  - Allowing for historical review/comparison of log entries

Using Java logging

- Import `java.util.logging`
- Get a `Logger` object
  - Use `getLogger(String)` to get a Logger object
  - The `String` names the logger
  - Typically, name the logger with the class name

```
import java.util.logging.*;
```

```
class HelloWorld {

  public static Logger logger =
    Logger.getLogger("HelloWorld");

}
```

[1] See https://docs.oracle.com/javase/8/docs/api/index.html?java/util/logging/Logger.html for additional log methods

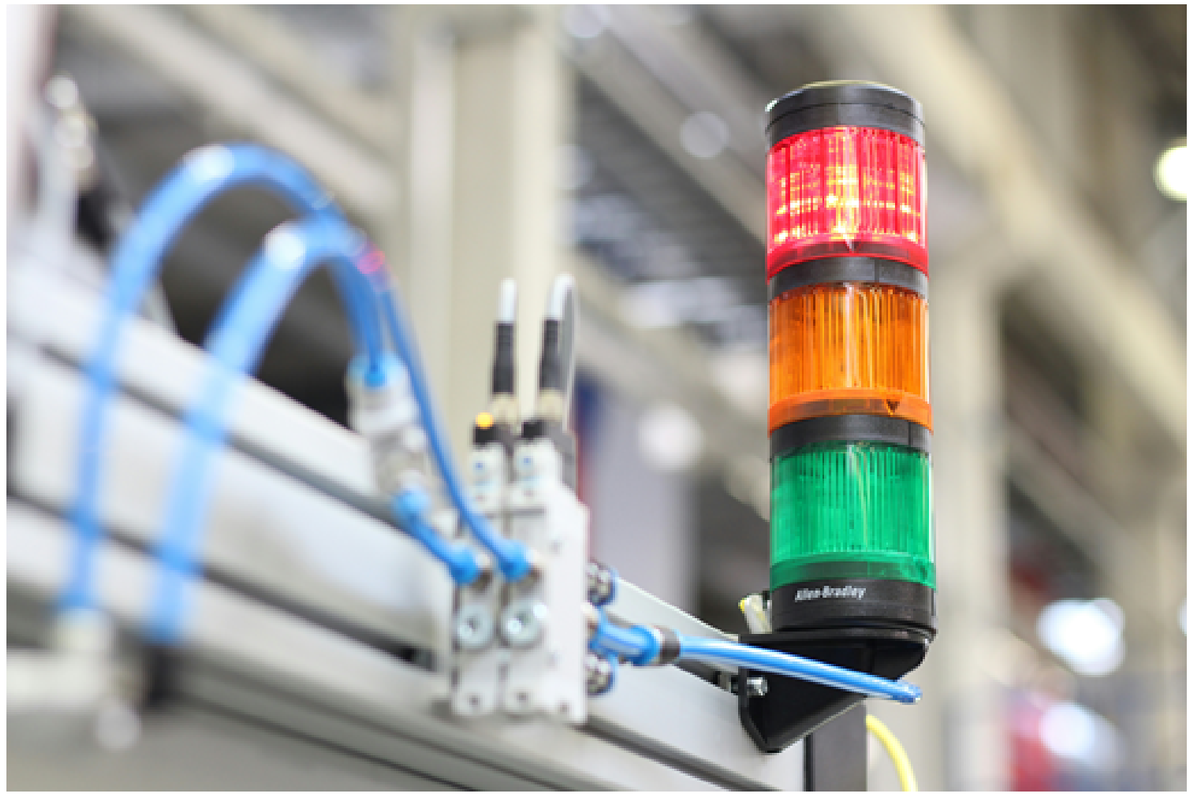
Writting log messages

```
public static void main (String[] args) {
  logger.log(Level.INFO,
             "This is informational");
  logger.log(Level.SEVERE,
             "This is a critical message");
}
```

```
Nov 26, 2024 12:59:29 PM HelloWorld main
INFO: This is informational
Nov 26, 2024 12:59:29 PM HelloWorld main
SEVERE: This is a critical message
```

Log levels

Level	Order
SEVERE	highest importance
WARNING	
INFO	
CONFIG	
FINE	
FINER	
FINEST	lowest importance



[1] Image courtesy https://unsplash.com/@vendavlog

Let's practice!

DATA TYPES AND EXCEPTIONS IN JAVA

Wrap Up

DATA TYPES AND EXCEPTIONS IN JAVA

Jim White
Java Developer

What we learned

- POJOs
  - Java's lightweight data suitcases
- Wrapper classes
  - Java's means to treat primitives like objects
- Using Java packages
  - How Java organizes code, and java.math as an example package
- Java's Collections Framework
  - Generic data structures for managing groups of objects
- Exception handling
  - Java's means of detecting and handling issues in an application
- Logging

To learn more

- Internet Sources
  - Baeldung.com: Website dedicated to Java topics and assistance
  - Oracle.com/java: The stewards of Java technology (Oracle) web site
  - dev.java: Java developer news, events, and other information
  - OpenJDK.org: Community dedicated to a free and open-source implementation of Java
- Books
  - *Head First Java* by Sierra et al
  - *Learn Java In One Day* by Jamie Chan
  - *Java for Beginners* by Brady Ellison
- Additional **DataCamp** classes and resources

Thank you!

DATA TYPES AND EXCEPTIONS IN JAVA